LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 07 – ABSTRACT & INTERFACE



Maysa Azhra Fauziyyah

103132430005

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

7.1 Kelas Abstract

Kata kunci abstract dipakai untuk class atau method yang belum punya implementasi. Method abstract hanya boleh dideklarasikan di dalam class abstract. Class abstract nggak bisa dibuat objeknya pakai new, jadi baru bisa dipakai lewat kelas turunannya, dan kelas turunan yang konkret wajib mengimplementasikan semua method abstract-nya. Contohnya di modul: Employee adalah class abstract dengan method abstract earnings(), lalu diturunkan ke SalariedEmployee, CommissionEmployee, dan HourlyEmployee yang masing-masing punya rumus gaji sendiri.

7.2 Interface

Kalau kelas mendefinisikan kemampuan sebuah objek, interface mendefinisikan method atau konstanta yang nantinya diimplementasikan oleh objek lain. Contohnya radio, TV, dan speaker sama-sama punya pengontrol volume, jadi ketiganya bisa mengimplementasikan interface yang sama. Interface di Java 8 punya beberapa batasan:

- semua atribut otomatis public static final (konstanta),
- hanya boleh berisi method abstract, default, dan static,
- nggak boleh ada constructor.

Interface juga membolehkan *multiple inheritance*, artinya satu kelas bisa mengimplementasikan lebih dari satu interface, padahal sebuah kelas cuma boleh punya satu kelas induk. Hubungan antara kelas dan interface di UML disebut *realization*, misalnya Invoice dan Employee yang mengimplementasikan Payable.

7.3 Default Method dan Static Method

Sebelum Java 8, interface cuma boleh berisi method abstract. Sejak Java 8, interface boleh punya method default dan static, supaya pengembang bisa menambah method baru ke interface tanpa mengubah kelas yang sudah mengimplementasikannya. Method default punya isi dan otomatis diwarisi kelas yang mengimplementasikan interface, jadi kelasnya nggak wajib menulis ulang. Method static juga punya isi, tapi dipanggil lewat nama interface-nya (misalnya MyInterface.anotherNewMethod()) dan nggak bisa ditimpa di kelas implementasi.

7.4 Perbedaan Abstract Class dan Interface

- Method di interface nggak punya body (di luar default dan static), sedangkan abstract class boleh punya method berisi.
- Interface cuma bisa mendefinisikan konstanta, sedangkan abstract class bisa mendeklarasikan variabel biasa.
- Interface nggak punya hubungan inheritance langsung dengan kelas tertentu, sedangkan abstract class bisa jadi turunan dari abstract class induknya.
- Interface memungkinkan pewarisan jamak (*multiple inheritance*), sedangkan abstract class tidak.

B. Guided

1. Guided-gd1

#Employee

```
package com.maysa.projectmodul07.guided.gd1;

public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName() {
        return name;
    }

    //Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

}
```

#SalariedEmployee

```
package com.maysa.projectmodul07.guided.gd1;

//Bentuk konkret: perannya jelas sebagai pegawai dengan skema gaji tetap
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }

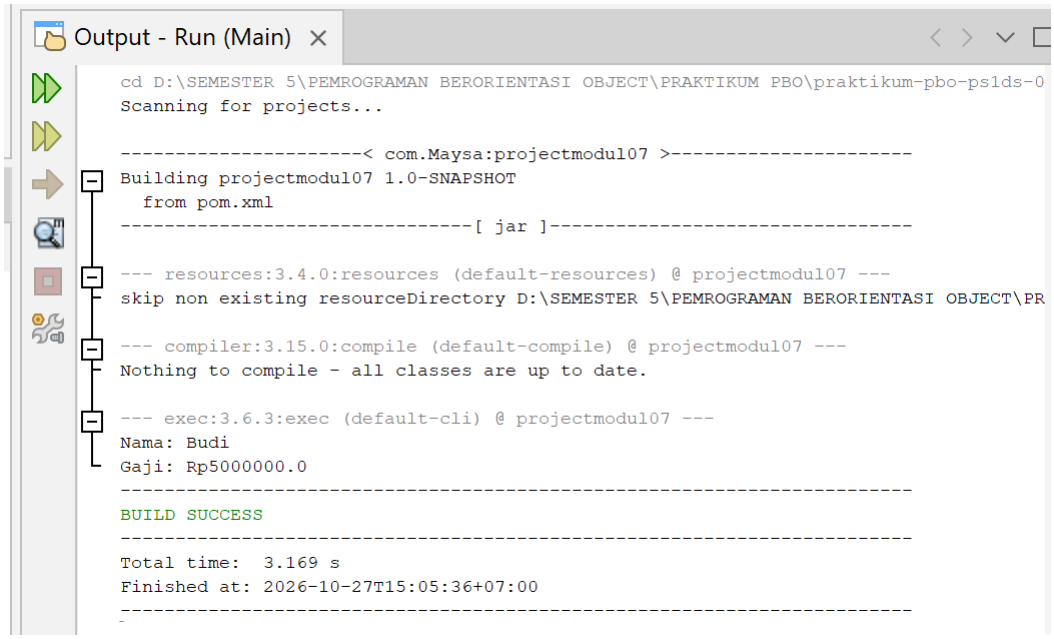
}
```

#Main

```
package com.maysa.projectmodul07.guided.gd1;

public class Main {
    public static void main(String[] args){
        SalariedEmployee salaried = new SalariedEmployee("Budi", 5000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output :



Kesimpulan:

Employee dibuat abstract dengan method earnings() tanpa isi, karena rumus gaji tiap jenis pegawai berbeda. SalariedEmployee sebagai turunan konkretnya wajib meng-override earnings() dan mengembalikan weeklySalary. Di Main, objek dibuat dari SalariedEmployee, bukan Employee, karena class abstract nggak bisa di-new. Atribut name yang private diisi lewat super(name) dan dibaca lewat getName().

2. Guided-gd2

```
#Employee
package com.maysa.projectmodul07.guided.gd2;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName() {
        return name;
    }
    //Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

    //Saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

#Invoice

```
package com.maysa.projectmodul07.guided.gd2;

//Benda mati non pegawai,
public class Invoice implements Payable{
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    //Hitung tagihan
    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }

}
```

#Payable

```
package com.maysa.projectmodul07.guided.gd2;

//Standar sistem: siapa pun yang mau ditagih oleh kasir wajib menyepakati method ini
public interface Payable {
    double getPaymentAmount();
}
```

#SalariedEmployee

```
package com.maysa.projectmodul07.guided.gd2;

//Bentuk konkret: perannya jelas sebagai pegawai dengan skema gaji tetap
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }

}
```

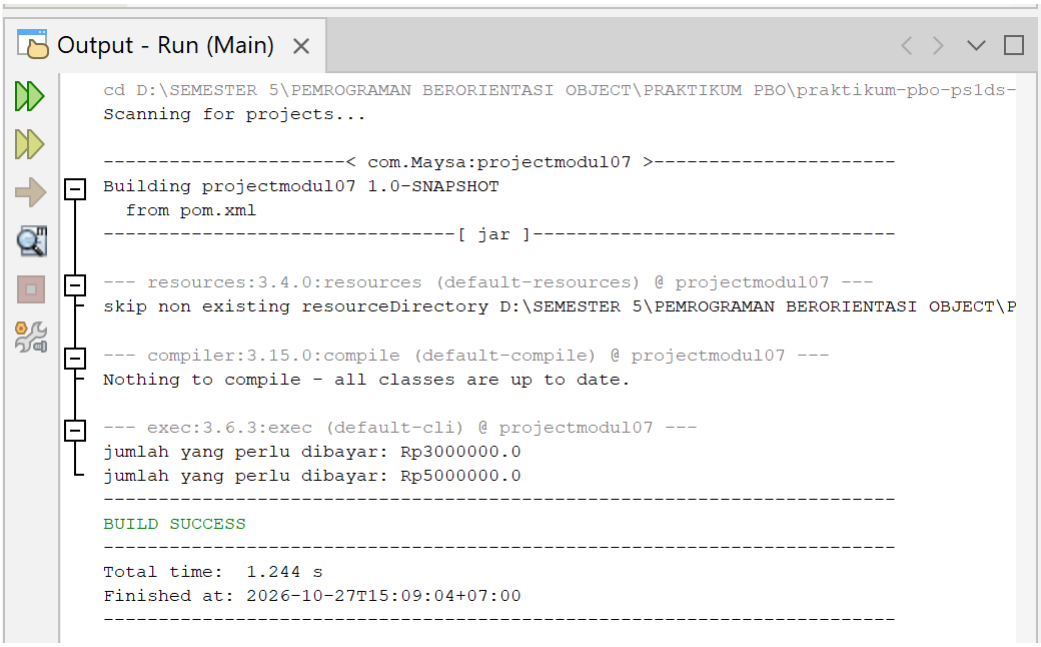
```
#Main
package com.maysa.projectmodul07.guided.gd2;

public class Main {
    public static void main(String[] args){
        Payable[] items = new Payable[2];

        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        // Dynamic dispatch = Java otomatis mengeksekusi rumus yang tepat sesuai objek
        for (Payable item : items){
            System.out.println("jumlah yang perlu dibayar: Rp" + item.getPaymentAmount());
        }
    }
}
```

Output:



Kesimpulan:

Invoice (benda mati) dan SalariedEmployee (pegawai) nggak sekeluarga, tapi sama-sama mengikuti kontrak Payable, jadi keduanya bisa disimpan dalam satu array Payable[]. SalariedEmployee ikut Payable lewat Employee yang meng-implement Payable, dan getPaymentAmount()-nya diarahkan ke earnings(). Saat perulangan, Java otomatis menjalankan rumus yang sesuai dengan objeknya (*dynamic dispatch*): Invoice 2 × 1500000 = 3000000, sedangkan pegawai 5000000

3. Guided-gd3

#Employee

```
package com.maysa.projectmodul07.guided.gd3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

    // Saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

#Invoice

```
package com.maysa.projectmodul07.guided.gd3;

// Benda mati non pegawai
public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    // Hitung tagihan
    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }
}
```

#Payable

```
package com.maysa.projectmodul07.guided.gd3;

public interface Payable {
    double getPaymentAmount();

    default void printReceipt() {
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    // Fungsi utilitas
    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

#SalariedEmployee

```
package com.maysa.projectmodul07.guided.gd3;

// Bentuk konkret: pegawai dengan skema gaji tetap
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

#Main

```
package com.maysa.projectmodul07.guided.gd3;

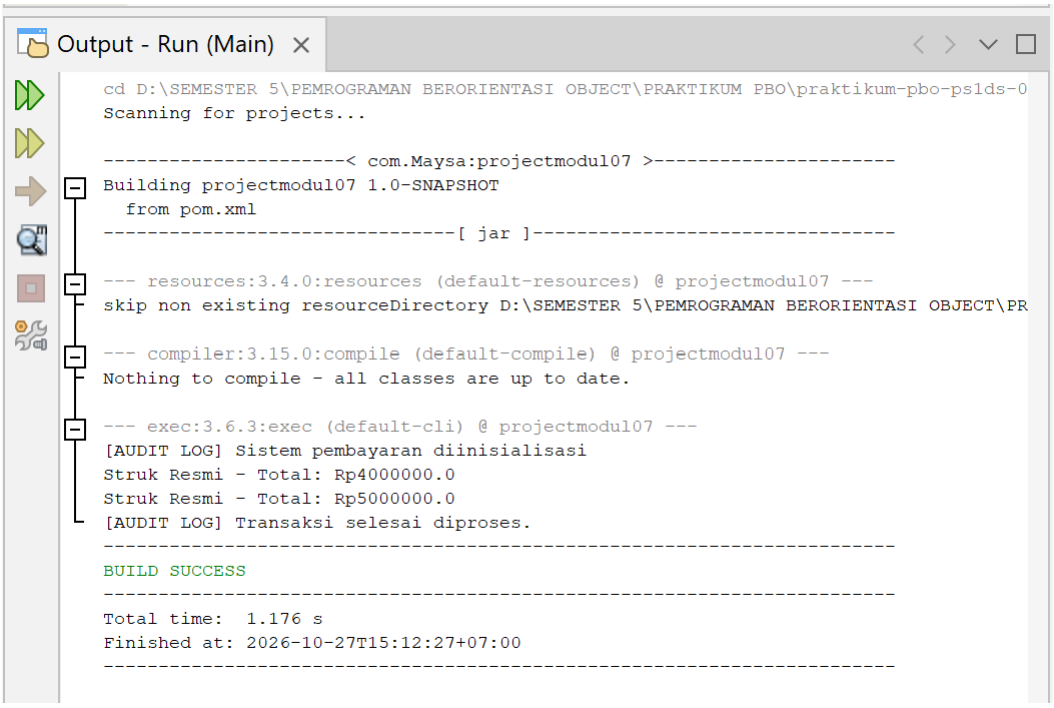
public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items) {
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");
    }
}
```

Output:



Kesimpulan:

printReceipt() ditulis sebagai default method di Payable, jadi Invoice dan SalariedEmployee bisa langsung memanggilnya tanpa menulis ulang. Hasilnya tetap sesuai objek masing-masing karena di dalamnya memanggil getPaymentAmount(): Invoice 2 × 2000000 = 4000000 dan pegawai 5000000. Sedangkan logTransaction() adalah static method yang dipanggil lewat Payable.logTransaction(...) tanpa perlu membuat objek.

C. Unguided

Soal Unguided — Modul 7: Pemrograman Berorientasi Objek

Abstraksi, Kontrak Interface, dan Polimorfisme Heterogen

Petunjuk Umum
Buatlah program Java berdasarkan skenario sistem kasir bioskop berikut. Rancang kelas dan antarmuka dengan mematuhi prinsip OOP yang telah dipelajari.

- **Struktur Berkas Pengerjaan (unguided/)** Seluruh kelas dan antarmuka wajib berada di dalam direktori unguided/ dengan rincian berkas berikut (satu file .java per kelas/interface):

```
unguided/
├── ItemTransaksi.java  (Interface)
├── TiketBioskop.java   (Abstract Class)
├── TiketReguler.java   (Concrete Subclass)
├── TiketVIP.java       (Concrete Subclass)
├── PaketSnack.java     (Concrete Class)
└── Main.java           (Kelas Eksekusi Utama)
```

Pastikan baris paling atas di setiap berkas mencantumkan package yang sesuai (contoh: package com.[nama_panggilan].unguided;).

- **Enkapsulasi:** Seluruh *field* (atribut) wajib berstatus private dan hanya boleh diakses melalui *getter* oleh kelas di luar dirinya.
- **Keyword this:** Gunakan parameter dengan nama yang sama dengan nama *field* pada *constructor*, sehingga wajib menggunakan kata kunci this secara eksplisit.
- **Koleksi Statis:** Dilarang menggunakan tipe data koleksi dinamis seperti ArrayList. Gunakan array statis standar.

Soal — Sistem Kasir Terpadu Bioskop (*Cinema POS*)
Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli **Tiket Nonton**, tetapi juga bisa membeli makanan/minuman (**Paket Snack F&B**).

Secara konsep sistem, **Tiket Bioskop** dan **Paket Snack** berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (*interface*), dan fitur modern Java 8+.

Tugas 1: Fondasi Kelas Abstrak (*Abstract Class*)
Buatlah hierarki kelas tiket bioskop dengan spesifikasi berikut:

1. Kelas Induk TiketBioskop (Wajib abstract)

- **Atribut (Private):** judulFilm (String), hargaDasar (double), jumlahKursi (int).
- **Konstruktor:** Menerima dan menginisialisasi ketiga atribut tersebut.
- **Method:** Buat *getter* untuk ketiga atribut.
- **Abstract Method:**
  - public abstract double hitungTotalBayar();
  - public abstract String getNamaTransaksi(); *(Beban perumusan kedua metode ini diserahkan kepada kelas turunannya agar output di struk nanti spesifik untuk tiap jenis tiket).*

2. Kelas Turunan TiketReguler

- **Atribut Tambahan (Private):** nomorStudio (int).
- **Konstruktor:** Menginisialisasi atribut induk via super(judulFilm, hargaDasar, jumlahKursi) dan atribut lokal nomorStudio.
- **Override Method:**
  - hitungTotalBayar(): Mengembalikan hasil hargaDasar * jumlahKursi.
  - getNamaTransaksi(): Mengembalikan string "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")".

3. Kelas Turunan TiketVIP

- **Atribut Tambahan (Private):** biayaFasilitasVIP (double). *(Catatan: Biaya fasilitas seperti sofa recliner ini dikenakan untuk **setiap kursi**).*
- **Konstruktor:** Menginisialisasi atribut induk via super(...) dan atribut lokal biayaFasilitasVIP.
- **Override Method:**
  - hitungTotalBayar(): Mengembalikan hasil (hargaDasar + biayaFasilitasVIP) * jumlahKursi.
  - getNamaTransaksi(): Mengembalikan string "Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) + ")".

Tugas 2: Kontrak Antarmuka Lintas Entitas (*Interface*)
Sistem kasir membutuhkan standar publik agar setiap barang bisa diproses.

1. Antarmuka ItemTransaksi Buatlah *interface* bernama ItemTransaksi dengan dua deklarasi kontrak (metode abstrak):

- double getNominalBayar();
- String getNamaTransaksi();

2. Integrasi Kelas TiketBioskop ke Antarmuka

- Tambahkan implements ItemTransaksi pada deklarasi kelas TiketBioskop.
- **Penting (Alasan Arsitektural):** Metode hitungTotalBayar() adalah logika internal spesifik tiket, sedangkan getNominalBayar() adalah kontrak publik kasir. Wujudkan metode getNominalBayar() di dalam TiketBioskop yang isinya sekadar me-*return* nilai dari hitungTotalBayar().
- *(Catatan: Anda tidak perlu menata ulang getNamaTransaksi() di sini karena ia sudah dideklarasikan sebagai metode abstrak pada Tugas 1).*

3. Kelas Baru: PaketSnack (Benda Non-Tiket) Buat kelas independen bernama PaketSnack yang **bukan** turunan TiketBioskop, tetapi mengimplementasikan ItemTransaksi.

- **Atribut (Private):** namaPaket (String), jumlahPorsi (int), hargaPerPorsi (double).
- **Konstruktor:** Menginisialisasi ketiga atribut.
- **Override Method:**
  - getNominalBayar(): Mengembalikan hasil jumlahPorsi * hargaPerPorsi.
  - getNamaTransaksi(): Mengembalikan string "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi + ")".

Tugas 3: Modernisasi Antarmuka Java 8+
Suntikkan fitur tambahan ke antarmuka ItemTransaksi tanpa mengharuskan kelas-kelas di atas dimodifikasi ulang:

1. Default Method Tambahkan metode default void cetakRincianItem() yang bertugas mencetak format baris struk ke layar. Panggil metode milik antarmuka (getNamaTransaksi dan getNominalBayar) untuk mengambil data secara dinamis. Format wajib: - [Nama Transaksi (Kuantitas x Harga Satuan)] | Total: Rp[Nominal Bayar]

2. Static Method Tambahkan metode static void catatLogKasir(String pesan) yang mencetak teks: [LOG SISTEM KASIR] [pesan]

Tugas 4: Pengujian Polimorfisme di Main
Tuliskan alur berikut di dalam *method* main:

1. **Uji Proteksi Abstraksi:** Tuliskan baris kode instansiasi TiketBioskop (misal: new TiketBioskop(...)), lalu jadikan baris tersebut sebagai **komentar** (//).
2. **Buka Kasir:** Panggil metode statis catatLogKasir("Membuka transaksi baru untuk Pelanggan #042."); langsung melalui nama antarmukanya.
3. **Buat Objek Transaksi:**
   - 1 objek TiketReguler (Film "Interstellar", harga dasar 40000, 2 kursi, Studio 3).
   - 1 objek TiketVIP (Film "Dune: Part Two", harga dasar 60000, 2 kursi, biaya VIP 25000).
   - 1 objek PaketSnack ("Combo Popcorn Caramel + 2 Soda", 1 porsi, harga 55000).
4. **Array Polimorfik:** Simpan ketiga objek tersebut ke dalam satu array heterogen bertipe ItemTransaksi[].
5. **Pemrosesan Dinamis (Loop):**
   - Gunakan perulangan (*for* atau *for-each*) terhadap array tersebut.
   - Di dalam perulangan, panggil *default method* cetakRincianItem() **lewat masing-masing objek/elemen array**.
   - Hitung dan jumlahkan nominal bayar keseluruhan belanjaan secara akumulatif.
6. **Tutup Kasir:** Tampilkan grand total seluruh belanjaan, lalu panggil kembali metode statis catatLogKasir("Transaksi #042 selesai diproses.");.

Contoh Output yang Diharapkan

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Jawaban :

1. Unguided - ItemTransaksi

```
package com.maysa.projectmodul07.unguided;

public interface ItemTransaksi {
    // Kontrak publik yang harus dipenuhi semua item kasir
    double getNominalBayar();

    String getNamaTransaksi();

    // Default method: dipakai langsung oleh semua kelas yang mengimplementasikan
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" + getNominalBayar());
    }
    // Static method: dipanggil lewat nama interface
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

2. Unguided - TiketBioskop

```
package com.maysa.projectmodul07.unguided;

public abstract class TiketBioskop implements ItemTransaksi {

    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    // Abstract method: rumusnya ditentukan oleh kelas turunan
    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    // Kontrak kasir dari ItemTransaksi, isinya cukup mengembalikan hitungTotalBayar()
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

3. Unguided - TiketReguler

```
package com.maysa.projectmodul07.unguided;

public class TiketReguler extends TiketBioskop {

    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
```

```
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

4. Unguided - TiketVIP

```
package com.maysa.projectmodul07.unguided;

public class TiketVIP extends TiketBioskop {

    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        // Biaya fasilitas VIP dikenakan untuk setiap kursi
        return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) +
")";
    }
}
```

5. Unguided - PaketSnack

```
package com.maysa.projectmodul07.unguided;

// Bukan turunan TiketBioskop, tetapi tetap bisa masuk kasir lewat ItemTransaksi
public class PaketSnack implements ItemTransaksi {

    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return namaPaket;
    }

    public int getJumlahPorsi() {
        return jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" +
hargaPerPorsi + ")";
    }
}
```

6. Unguided - Main

```
package com.maysa.projectmodul07.unguided;

public class Main {

    public static void main(String[] args) {
```

```
        // 1. Uji proteksi abstraksi: kelas abstract tidak bisa diinstansiasi
        // TiketBioskop tiketError = new TiketBioskop("Film", 50000, 1);  // ERROR: TiketBioskop is abstract

        // 2. Buka kasir (static method dipanggil lewat nama interface)
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");

        // 3. Buat objek transaksi
        TiketReguler tiketReguler = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack paketSnack = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1, 55000);

        // 4. Array polimorfik: tiga jenis objek berbeda dalam satu array bertipe ItemTransaksi
        ItemTransaksi[] keranjang = {tiketReguler, tiketVIP, paketSnack};

        // 5. Pemrosesan dinamis
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        // 6. Tutup kasir
        System.out.println("------------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);

System.out.println("============================================================");
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Penjelasan source code:

Program ini membuat sistem kasir bioskop yang bisa memproses tiket dan snack dalam satu keranjang, padahal keduanya berasal dari hierarki kelas yang berbeda. Semua kelas ada di satu package com.maysa.projectmodul07.unguided.

- **`ItemTransaksi` (interface).** Berisi kontrak `getNominalBayar()` dan `getNamaTransaksi()`. Ada juga `default` method `cetakRincianItem()` yang mencetak satu baris struk dengan memanggil kedua method kontrak itu, dan `static` method `catatLogKasir()` yang dipanggil lewat nama interface. Karena `default`, semua kelas yang mengimplementasikannya langsung punya `cetakRincianItem()` tanpa mengubah kodenya.

- **`TiketBioskop` (abstract class).** Menyimpan `judulFilm`, `hargaDasar`, dan `jumlahKursi` sebagai `private` dengan getter. Punya dua abstract method, `hitungTotalBayar()` dan `getNamaTransaksi()`, yang rumusnya diserahkan ke kelas turunan. Kelas ini mengimplementasikan `ItemTransaksi`, dan `getNominalBayar()` cukup me-return `hitungTotalBayar()`. Karena `abstract`, kelas ini nggak bisa di-`new`.
- **`TiketReguler` dan `TiketVIP`.** Keduanya `extends TiketBioskop` dan memanggil `super(...)` di constructor. `TiketReguler` menghitung harga × kursi, sedangkan `TiketVIP` menghitung (harga + biaya VIP) × kursi karena biaya fasilitas dikenakan per kursi. Karena atribut induk `private`, rumusnya memakai getter.
- **`PaketSnack`.** Bukan turunan `TiketBioskop`, tapi mengimplementasikan `ItemTransaksi` langsung, dengan `getNominalBayar()` = porsi × harga per porsi.
- **`Main`.** Baris `new TiketBioskop(...)` dibuat jadi komentar sesuai soal. Tiga objek dari kelas berbeda disimpan dalam satu array `ItemTransaksi[]`, lalu perulangan `for-each` memanggil `cetakRincianItem()` dan menjumlahkan `getNominalBayar()` tiap elemen.

Semua atribut private dan diakses lewat getter, constructor memakai parameter bernama sama dengan field (jadi pakai this), dan array yang dipakai adalah array statis biasa, bukan ArrayList.

Output:

```
----------------------< com.Maysa:projectmodul07 >----------------------
Building projectmodul07 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ projectmodul07 ---
skip non existing resourceDirectory D:\SEMESTER 5\PEMROGRAMAN BERORIENTASI OBJECT\PRAKTIKUM PBO\prakti

--- compiler:3.15.0:compile (default-compile) @ projectmodul07 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ projectmodul07 ---
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  0.840 s
Finished at: 2026-11-01T00:13:41+07:00
------------------------------------------------------------------------
```

Penjelasan Output:
Walaupun ketiga objek disimpan dalam array bertipe ItemTransaksi, setiap pemanggilan method menjalankan versi milik kelas aslinya (polimorfisme), makanya format tiap baris berbeda. Perhitungannya:

- Tiket Reguler: 40000 × 2 kursi = 80000.
- Tiket VIP: (60000 + 25000) × 2 kursi = 85000 × 2 = 170000. Harga satuan di struk sudah 85000 karena sudah termasuk biaya VIP.
- Paket Snack: 1 × 55000 = 55000.
- Grand total: 80000 + 170000 + 55000 = 305000.

D. Kesimpulan

Dari bagian guided, tiga konsep dipraktikkan lewat contoh terpisah. Pada abstract class (gd1), Employee menetapkan kerangka dengan method abstract earnings(), dan SalariedEmployee sebagai turunan konkretnya wajib mengisi rumusnya, sementara Employee sendiri nggak bisa dibuat objeknya. Pada interface (gd2), Invoice dan SalariedEmployee yang nggak sekeluarga tetap bisa disimpan dalam satu array Payable[] karena sama-sama mengikuti kontrak getPaymentAmount(), dan Java menjalankan rumus yang sesuai dengan objeknya. Pada default dan static method (gd3), printReceipt() bisa dipakai semua kelas tanpa kode ulang, dan logTransaction() dipanggil langsung lewat nama interface.

Pada bagian unguided, ketiganya digabung dalam sistem kasir bioskop. TiketBioskop sebagai abstract class menetapkan kerangka tiket dan menyerahkan cara hitung ke TiketReguler dan TiketVIP, sedangkan ItemTransaksi menyatukan tiket dan PaketSnack yang hierarkinya berbeda dalam satu kontrak. Hasilnya, ketiganya bisa disimpan dalam satu array dan diproses dengan perulangan yang sama, dengan total transaksi Rp305000.0 sesuai yang diminta soal.

Dari keseluruhan praktikum, dapat disimpulkan bahwa abstract class dipakai ketika beberapa kelas punya kerangka yang sama tapi cara hitung atau perilaku yang berbeda, sedangkan interface dipakai untuk menyatukan kelas-kelas yang nggak sekeluarga dalam satu kontrak. Dengan begitu, tujuan praktikum untuk memahami abstract class dan interface serta perbedaannya sudah tercapai.